Como a Comunidade Testa Sistemas RAG e Agentes de LLM? Evidências da Literatura Cinza e de Repositórios de Código

Icaro Santos de Oliveira
Universidade Estadual do Ceará (UECE)
Fortaleza, CE, Brazil
icaro.santos@aluno.uece.br

João Matheus de Lima Alves
Universidade Estadual do Ceará (UECE)
Fortaleza, CE, Brazil
joaomatheus.lima@aluno.uece.br

Ismayle de Sousa Santos
Universidade Estadual do Ceará (UECE)
Fortaleza, CE, Brazil
ismayle.santos@uece.br

RESUMO

Sistemas baseados em grandes modelos de linguagem (LLMs) com geração aumentada por recuperação (RAG) e agentes que escolhem ferramentas e executam planos chegaram à produção antes de existir um corpo consolidado de práticas de teste. Surveys acadêmicos catalogam métricas e *benchmarks*, mas não mostram o que a comunidade de desenvolvimento de fato adota. Este artigo relata um estudo de métodos mistos que combina literatura cinza e mineração de repositórios sobre 2.916 itens de seis fontes, coletados por APIs oficiais, com 1.890 detecções automáticas de ferramentas e métodos de teste. Cada ferramenta é classificada em três níveis de evidência (menção, uso declarado e adoção sustentada em integração contínua), e os *frameworks* do sistema sob teste ficam fora dos rankings. Discurso e prática divergem. O Ragas lidera as menções (46 unidades) e o uso declarado (21), mas a adoção sustentada concentra-se em *benchmarks* de agentes e em ferramentas de segurança, que somam 64% das ocorrências em integração contínua e apenas 17% das menções; ferramentas de rastreamento, como o LangSmith, são muito citadas e não têm nenhuma adoção sustentada. A concordância entre os rankings por uso declarado e por adoção é negativa ($\tau_b = -0{,}38$). LLM como juiz é o método mais frequente (175 unidades); oráculos pseudoautomáticos (51%) e de referência (49%) predominam, e oráculos derivados aparecem em apenas 4% das unidades com algum sinal de oráculo. Discutimos as implicações para praticantes, pesquisadores e criadores de ferramentas.

PALAVRAS-CHAVE

Teste de Sistemas Inteligentes, Geração Aumentada por Recuperação, Agentes de LLM, Literatura Cinza, Revisão Multivocal, Mineração de Repositórios de Software

1 INTRODUÇÃO

Sistemas que combinam LLMs com recuperação de documentos (RAG) [18, 26] e sistemas em que um LLM escolhe ferramentas e executa planos (agentes) passaram a ser construídos em escala a partir de 2023. Testá-los é difícil: as saídas não são determinísticas, o critério de correção costuma ser vago e as falhas podem nascer na recuperação, na geração, na escolha de uma ferramenta ou na interação entre passos [1, 5]. É uma versão aguda do problema do oráculo [2], já discutido no teste de sistemas de aprendizado de máquina [36].

A literatura acadêmica sobre o tema concentra-se em métricas e *benchmarks* de avaliação [6, 34, 35]. O que as equipes de desenvolvimento de fato usam para testar esses sistemas aparece em outro lugar: em repositórios que importam bibliotecas de avaliação como Ragas [17], DeepEval [9] e promptfoo [29], em perguntas de fóruns técnicos, em discussões de agregadores e em postagens de blogs. Essa literatura cinza já serviu para responder que ferramentas de automação de teste os praticantes escolhem [31], mas tem dois vieses conhecidos: uma ferramenta muito citada não é necessariamente usada, e popularidade em plataformas sociais não é sinônimo de adoção [4, 22].

Este artigo investiga como a comunidade testa sistemas RAG e agentes de LLM, separando o que se *discute* do que se *adota*. Combinamos revisão de literatura cinza [19] e mineração de repositórios em um corpus de 2.916 itens de seis fontes, coletados por APIs oficiais com uma ferramenta de mineração descrita e validada em trabalho complementar [12]. Cada ferramenta detectada recebe um nível de evidência: **menção** (N1), **uso declarado** (N2: importação, configuração ou dependência) ou **adoção sustentada** (N3: invocação em integração contínua, CI, de um repositório ativo e com mais de um contribuidor). As questões de pesquisa são:

QP1. Quais ferramentas de teste e avaliação a comunidade mais usa em sistemas RAG e em agentes, e com que força de evidência?
QP2. Quais técnicas de teste e quais tipos de oráculo a comunidade emprega?
QP3. Quais ferramentas e técnicas geram mais engajamento, e o engajamento acompanha a adoção?

Duas questões complementares, sobre os modos de falha relatados (QP4) e sobre o que se transfere de RAG para agentes (QP5), dependem da codificação manual, em andamento, e aparecem aqui como prévia.

Os principais achados são: (i) a adoção sustentada em CI se concentra em *benchmarks* de agentes e em ferramentas de segurança, enquanto as bibliotecas de avaliação de RAG dominam as menções e o uso declarado; (ii) os rankings por menção e por adoção não concordam, e o ranking por uso declarado chega a se opor ao de adoção; (iii) ferramentas de rastreamento são muito citadas e não têm adoção sustentada; (iv) LLM como juiz é o método dominante, com poucos oráculos derivados ou humanos. Os dados e os scripts de análise são abertos, e cada número do artigo é rastreável até o relatório que o gerou.

A Seção 2 apresenta a fundamentação; a Seção 3, o método; a Seção 4, os resultados por questão; a Seção 5 discute implicações; a Seção 6 trata das ameaças à validade; a Seção 7 posiciona o trabalho; e a Seção 8 conclui.

2 FUNDAMENTAÇÃO

2.1 Teste de Sistemas RAG e de Agentes

Em um sistema RAG, a resposta depende do que foi recuperado. Barnett et al. [1] identificam sete pontos de falha, como conteúdo ausente na base, documento relevante fora do *top-k* e resposta não

extraída do contexto. A avaliação recorre a métricas como fidelidade e revocação do contexto, popularizadas pelo Ragas [17], e a LLMs usados como juízes [37]. Em agentes, além da resposta final, importa a trajetória: escolha e argumentos de ferramentas, terminação, efeitos colaterais e recuperação de erros. Em sistemas multiagentes, somam-se falhas de especificação e de coordenação [5]. Para esse cenário surgiram *benchmarks* como SWE-bench [21], WebArena [38], AgentBench [27], τ-bench [33] e AgentDojo [13], além de ferramentas de sondagem de segurança como o garak [14].

2.2 Oráculos de Teste

O oráculo decide se uma saída está correta [2]. Adotamos uma escada de cinco tipos:

- **especificado:** asserções sobre a saída, como formato ou chamada de ferramenta esperada;
- **derivado:** relações entre execuções, como no teste metamórfico e no baseado em propriedades;
- **referência:** comparação com um conjunto *golden* ou com uma execução anterior, como na regressão de avaliação;
- **pseudoautomático:** um LLM julga a saída;
- **humano:** uma pessoa julga a saída.

Sistemas estocásticos empurram a prática para os degraus mais caros ou menos confiáveis, e saber onde ela se concentra orienta praticantes e pesquisadores.

2.3 Literatura Cinza e Mineração de Repositórios

Revisões multivocais incorporam literatura cinza à evidência acadêmica, com critérios próprios de qualidade [19]. Minerar o GitHub exige cuidado com projetos pessoais, inativos ou de exemplo [22], e estrelas medem atenção, não uso [4]. Por isso o estudo não toma contagens brutas como adoção: exige sinais de uso no código e de manutenção ativa.

3 MÉTODO

3.1 Desenho do Estudo

É um estudo de métodos mistos. A parte quantitativa vem das detecções automáticas de ferramentas e métodos, com nível de evidência, e das métricas de engajamento; a qualitativa, da codificação manual dos itens incluídos, em andamento. O estudo segue as diretrizes de revisão multivocal [19], o PRISMA 2020 para o fluxo de seleção [28] e os padrões empíricos da ACM SIGSOFT para mineração de repositórios e revisões sistemáticas [30]. A Tabela 1 relaciona as questões às análises e proposições. Como os autores acompanharam coletas preliminares, todos os testes de proposição são **exploratórios**.

3.2 Corpus

O corpus vem da execução a2a8 (29/09/2026) do protocolo do estudo, coletada pelo MSR-Kit [11, 12] em seis fontes: GitHub (repositórios, código e *issues*), Stack Exchange, Hugging Face, Hacker News, dev.to e feeds RSS do Medium e de blogs. Todas as fontes foram acessadas por APIs oficiais ou feeds públicos, sem *scraping*. Reddit, X, LinkedIn e Discord ficaram de fora por exigirem aprovação prévia, acesso pago ou autorização de administradores.

Tabela 1: Questões, análises e proposições

QP	Análise	Proposição
QP1	pirâmide N3, N2 e N1; recorte RAG × agentes; coocorrência	P1: ranking por menção difere do ranking por adoção (Kendall τ_b)
QP2	métodos por sistema; escada de oráculos	H2: a maioria dos oráculos é atômica (requer codificação)
QP3	engajamento normalizado por fonte	P2: engajamento e adoção são fracamente correlacionados
QP4	modos de falha × ferramentas e métodos	requer codificação
QP5	transferência de RAG para agentes	requer codificação

Tabela 2: Coleta por fonte (execução a2a8). Itens após a deduplicação.

Fonte	Consultas	Requisições	Itens
GitHub	324	364	2.459
Hacker News	30	30	202
Hugging Face	30	77	122
Stack Exchange	30	146	117
dev.to	1	11	10
RSS (Medium e blogs)	1	14	6
Total	416	642	2.916

A busca usou um léxico controlado de 20 termos em inglês, que combina o sistema (RAG, LLM, agente) com teste ou avaliação. Ele cobre o vocabulário da prática (*LLM as a judge*, *golden dataset*, *trajectory evaluation*) e o acadêmico (*metamorphic testing*, *test oracle*); há também 10 termos em português. No GitHub, consultas adicionais buscam importações de bibliotecas de avaliação, arquivos de configuração e a invocação de cada ferramenta do catálogo em *workflows* de CI. A janela vai de 01/2023 a 08/2026, e cada consulta trouxe no máximo 10 itens, para espalhar a amostra pelo léxico. Um catálogo de 41 ferramentas e 10 métodos orienta a extração: 33 são ferramentas de teste e 8 são *frameworks* do sistema sob teste.

A coleta identificou 3.668 registros em 416 consultas (Tabela 2). Após os filtros do protocolo (496 registros) e a deduplicação, 2.916 itens seguiram para a triagem, 84% deles do GitHub. Os 300 repositórios com maior potencial de evidência, a começar pelos que tinham invocações em CI, foram enriquecidos, sem erros, com sinais de CI, dependências, commits e contribuidores. A extração produziu 1.890 detecções, agregadas em 1.835 unidades de análise; 10 menções a nomes ambíguos foram descartadas por falta de contexto.

3.3 Unidades de Análise e Níveis de Evidência

A unidade de análise é um repositório do GitHub, com todos os seus itens fundidos, ou um item isolado nas demais fontes. Cada unidade conta uma vez por ferramenta, no maior nível atingido:

- **N1:** menção em texto;

- **N2:** uso declarado, isto é, importação encontrada na busca de código, arquivo de configuração ou pacote no manifesto de dependências;
- **N3:** adoção sustentada, isto é, invocação da ferramenta em um *workflow* de CI de um repositório com commits em pelo menos dois meses da janela e pelo menos dois contribuidores.

Os *frameworks* do sistema sob teste (LangChain, LangGraph, LlamaIndex e afins) não entram no ranking de ferramentas de teste: são reportados como contexto, isto é, sobre quais pilhas os testes são feitos. Na base de detecções, o rótulo de sistema (RAG, agente ou ambos) vem dos conceitos do léxico casados em cada unidade.

3.4 Triagem e Codificação

A triagem aplica os critérios do protocolo. Incluem-se itens que tratam de teste ou avaliação com veredito de sistemas RAG ou agênticos, públicos, em inglês ou português e dentro da janela. Excluem-se, entre outros, itens que usam LLMs para testar outro software, que tratam só do modelo isolado e conteúdo promocional sem detalhe técnico.

A triagem segue duas trilhas: uma ordenada por relevância, em lotes de 25, até a saturação dos códigos, e uma amostra aleatória estratificada por fonte e sistema, que permite estimar proporções com intervalos de Wilson [32]. Pelo menos 20% dos itens passam por dois avaliadores, com κ de Cohen [8] interpretado segundo Landis e Koch [25]. Os incluídos são codificados por um formulário que registra sistema, nível de evidência, ferramentas, métodos, modos de falha (13 de RAG e 12 de agentes), tipo de oráculo e agregação. Os modos de falha relatados passam por síntese temática [10].

Uma amostra estratificada de 200 detecções foi separada para medir a precisão do extrator. Como a triagem e a codificação estão em andamento, os resultados a seguir usam a base de detecções automáticas.

3.5 Análises

QP1. Contamos as unidades por ferramenta em cada nível, com o N3 como ranking principal. A lacuna entre discurso e prática é medida pela correlação de Kendall τ_b [23] entre os números de unidades por nível, sobre as ferramentas de teste com contagem disponível no relatório da execução. A P1 é lida como sustentada quando $\tau_b < 0{,}5$. A coocorrência entre ferramentas, métodos e *frameworks* usa Jaccard e *lift*.

QP2. Contamos os métodos por sistema e posicionamos cada unidade na escada de oráculos pelo oráculo típico dos métodos detectados, uma aproximação até que o tipo de oráculo seja codificado.

QP3. O engajamento é o percentil da métrica principal de cada item (reações, votos ou curtidas) dentro de cada par fonte × tipo de item, de modo que contagens brutas nunca são comparadas entre fontes. Só entram itens discursivos com pelo menos três meses entre a criação e a coleta. As ferramentas são comparadas por Kruskal–Wallis [24], Dunn [15] com correção de Holm [20] e δ de Cliff [7]. A relação com a adoção é medida por Spearman, com intervalos por *bootstrap* [16].

Descoberta aberta. Um passe com LDA [3] e k-means sobre o texto dos itens procura termos frequentes fora do catálogo, para reduzir o viés de buscar só o que já se conhece.

Os números vêm dos relatórios de análise da execução (`msrkit extract` e `msrkit analyze`), e as correlações de Kendall foram calculadas sobre as contagens desses relatórios. Limiares e parâmetros são decisões provisórias registradas no ADR-046 do repositório; como os dados já eram conhecidos, as análises são exploratórias. Nas próximas rodadas, o script `analysis/artigo2.py` gera todos os números a partir do pacote de reprodução.

4 RESULTADOS

Salvo indicação, os resultados vêm da base de detecções automáticas da execução a2a8, antes da triagem manual, e contam unidades de análise.

4.1 QP1: Ferramentas Mais Usadas

A Tabela 3 apresenta a pirâmide de evidência das 24 ferramentas de teste com contagem no relatório da execução. As demais ferramentas do catálogo aparecem em até 9 unidades e não têm adoção sustentada. Foram 97 ocorrências de adoção sustentada (N3) em 21 ferramentas. O topo é formado por WebArena, Guardrails AI e AgentDojo (8 unidades cada), seguidos de τ-bench e garak (7). O uso declarado (N2) é liderado por Ragas (21 unidades) e DeepEval (19), e as menções (N1) por Ragas (46), LangSmith (41) e lm-evaluation-harness (26).

A pirâmide se inverte entre famílias. *Benchmarks* de agentes e ferramentas de guardrails e segurança respondem por 64% das ocorrências em N3 (62 de 97), mas só por 17% das menções (35 de 211) e 25% do uso declarado (32 de 128). As bibliotecas de avaliação de LLMs e RAG concentram 50% das menções e 56% do uso declarado, mas 36% da adoção sustentada. As ferramentas de rastreamento e observabilidade (LangSmith, Langfuse e Arize Phoenix) somam 34% das menções e nenhuma ocorrência em N3.

Recorte RAG × agentes (QP1.2). Entre as ferramentas com rótulo disponível, Ragas e DeepEval aparecem tanto em unidades de RAG quanto de agentes (39 e 29; 16 e 17). Os *benchmarks* de agentes aparecem sobretudo em unidades de agentes, como o SWE-bench (13 contra 6) e o AgentBench (6 contra 1). Muitas unidades de código não têm rótulo de sistema, porque não casam termos do léxico; por isso a soma das colunas fica abaixo do total.

Discurso × prática (QP1.3). Sobre as 24 ferramentas, a concordância entre o ranking por menções e o ranking por adoção sustentada é $\tau_b = -0{,}18$ ($p = 0{,}24$). Entre uso declarado e adoção, a concordância é negativa, $\tau_b = -0{,}38$ ($p = 0{,}017$); entre menção e uso declarado, é positiva, $\tau_b = 0{,}46$ ($p = 0{,}004$). Os dois primeiros valores ficam abaixo do limiar de 0,5 e sustentam a P1. Menção e uso declarado andam juntos, mas nenhum dos dois prevê a adoção em CI.

A Figura 1 mostra a dispersão. Acima da diagonal, muito citadas e pouco adotadas em CI, estão LangSmith (41 menções, nenhuma unidade N3), Langfuse (20 e 0), Arize Phoenix (10 e 0), Ragas (46 e 5) e lm-evaluation-harness (26 e 4). Abaixo dela, adotadas além do que são citadas, estão Guardrails AI e AgentDojo (0 menções e 8 unidades N3), τ-bench (1 e 7), autoevals e Giskard (1 e 6) e PyRIT (0 e 4).

Pilhas e contexto. Os *frameworks* mais presentes são LangChain (146 unidades: 97 com menção e 49 com uso declarado), LangGraph (67), LlamaIndex (34), DSPy (23), CrewAI (22) e AutoGen (12). Por

Tabela 3: Pirâmide de evidência das ferramentas de teste: unidades por maior nível de evidência (execução a2a8, base de detecções). *Frameworks* do sistema sob teste excluídos. RAG e Agente contam unidades com o rótulo; uma unidade pode ter os dois ou nenhum, e "—" indica rótulo não disponível no relatório. Ordenada por N3, N2 e N1.

Ferramenta	Família	N3	N2	N1	Total	RAG	Agente
WebArena	agentes	8	2	4	14	1	3
Guardrails AI	guardrails/segurança	8	2	0	10	0	1
AgentDojo	agentes	8	1	0	9	0	3
τ-bench	agentes	7	2	1	10	2	1
garak	guardrails/segurança	7	1	3	11	1	3
autoevals	avaliação LLM/RAG	6	3	1	10	1	3
Giskard	avaliação LLM/RAG	6	2	1	9	—	—
Ragas	avaliação LLM/RAG	5	21	46	72	39	29
SWE-bench	agentes	5	5	15	25	6	13
NeMo Guardrails	guardrails/segurança	5	3	4	12	1	3
BFCL	agentes	5	2	4	11	1	2
DeepEval	avaliação LLM/RAG	4	19	22	45	16	17
lm-evaluation-harness	avaliação LLM/RAG	4	7	26	37	5	10
PyRIT	guardrails/segurança	4	3	0	7	—	—
agentevals	agentes	3	4	1	8	—	—
TruLens	avaliação LLM/RAG	3	4	0	7	—	—
promptfoo	avaliação LLM/RAG	2	9	8	19	1	4
AgentBench	agentes	2	7	3	12	1	6
continuous-eval	avaliação LLM/RAG	2	5	0	7	—	—
UpTrain	avaliação LLM/RAG	2	0	1	3	—	—
RAGChecker	avaliação LLM/RAG	1	2	0	3	—	—
LangSmith	rastreamento	0	12	41	53	34	38
Langfuse	rastreamento	0	10	20	30	14	18
Arize Phoenix	rastreamento	0	2	10	12	6	11
Total		97	128	211	436		

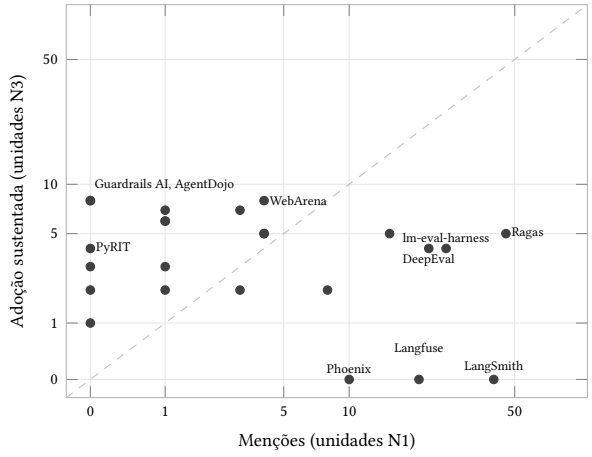


Figura 1: Menções (N1) × adoção sustentada (N3) das 24 ferramentas de teste, em unidades e escala logarítmica (os eixos somam 1 para exibir zeros). A linha tracejada marca contagens iguais. Execução a2a8, base de detecções.

desenho do catálogo, nenhum deles é buscado em CI. A Tabela 4 lista pares com *lift* alto, que indicam combinações usadas juntas. DeepEval e Ragas coocorrem 9,6 vezes mais que o esperado ao acaso. As ferramentas de rastreamento acompanham os *frameworks*: LangChain com LangSmith tem o maior *lift* da tabela (10,7). A avaliação de trajetória aparece associada ao LangGraph (6,5).

4.2 QP2: Técnicas e Oráculos

A Tabela 5 apresenta os métodos detectados. LLM como juiz é o mais frequente, em 175 unidades, seguido de conjunto *golden* (94), regressão de avaliação (88), *red teaming* (51) e teste por asserção (48). A avaliação de trajetória, específica de agentes, aparece em 21 unidades, o dobro em unidades de agentes (14) do que em RAG (7). Teste metamórfico, teste baseado em propriedades e geração sintética de testes aparecem em até 9 unidades cada.

Na escada de oráculos (Tabela 6), 51% das unidades com algum sinal de oráculo usam oráculo pseudoautomático e 49% de referência; uma unidade pode ter mais de um. O oráculo especificado aparece em 14%, o humano em 5% e o derivado em 4% (13 unidades). Em RAG e em agentes, o pseudoautomático é ainda mais frequente (64% e 60%). O teste de H2, sobre a agregação dos oráculos, depende da codificação manual.

4.3 QP3: Engajamento

A análise de engajamento usa as métricas de cada item (votos, reações e curtidas) e o pacote de reprodução da execução. Nesta versão, ela ainda não foi executada sobre o corpus: será reportada com o script de análise e a execução oficial congelada do protocolo. O passe de descoberta já indica a matéria-prima da QP3 e da QP4. Dos 2.094 documentos com texto suficiente, um agrupamento de

Tabela 4: Pares com *lift* alto entre ferramentas de teste (F), métodos (M) e *frameworks* do sistema sob teste (SUT), selecionados entre os 30 pares ferramenta–método e os 15 pares ferramenta–ferramenta mais frequentes do relatório. *Lift* sobre as 1.835 unidades (execução a2a8, base de detecções).

Tipo	Par	Juntas	Jaccard	Lift
F × F	DeepEval + Ragas	17	0,17	9,6
F × F	Langfuse + Ragas	8	0,09	6,8
F × M	garak + *red teaming*	3	0,05	9,8
F × M	promptfoo + *red teaming*	3	0,04	5,7
F × M	lm-evaluation-harness + teste por asserção	5	0,06	5,2
F × M	Ragas + conjunto *golden*	14	0,09	3,8
F × M	Ragas + LLM como juiz	18	0,08	2,6
F × M	DeepEval + LLM como juiz	10	0,05	2,3
SUT × F	LangChain + LangSmith	45	0,29	10,7
SUT × F	LangGraph + LangSmith	18	0,18	9,3
SUT × F	LangGraph + DeepEval	9	0,09	5,5
SUT × F	LangChain + Ragas	21	0,11	3,7
SUT × M	LangGraph + avaliação de trajetória	5	0,06	6,5
SUT × M	DSPy + LLM como juiz	10	0,05	4,6

Tabela 5: Métodos de teste por sistema, em unidades, com o oráculo típico de cada um (execução a2a8, base de detecções). Os demais métodos do catálogo aparecem em até 9 unidades.

Método	Oráculo típico	Unid.	RAG	Agente
LLM como juiz	pseudoaut.	175	86	103
conjunto *golden*	referência	94	42	41
regressão de avaliação	referência	88	34	54
red teaming	—	51	16	31
teste por asserção	especificado	48	27	34
avaliação de trajetória	—	21	7	14
avaliação humana	humano	17	9	8

Tabela 6: Escada de oráculos: unidades e percentual sobre as unidades com algum sinal de oráculo em cada sistema. Oráculo inferido do método detectado (aproximação); execução a2a8, base de detecções.

Oráculo	Todos	RAG	Agente
Pseudoautomático	175 (51%)	86 (64%)	103 (60%)
Referência	166 (49%)	63 (47%)	81 (47%)
Especificado	48 (14%)	27 (20%)	34 (20%)
Humano	17 (5%)	9 (7%)	8 (5%)
Derivado	13 (4%)	6 (4%)	6 (3%)

254 reúne relatos de *bug* com erro e reprodução. Outro, com 458 documentos, concentra discussões sobre avaliação de LLMs, RAG e agentes.

4.4 Descoberta Aberta e Ruído

O LDA (12 tópicos) e o k-means (12 grupos) sinalizaram 125 termos candidatos fora do catálogo. Entre eles há ferramentas a avaliar, como o MLflow, que aparece agrupado com o Opik em discussões de rastreamento e avaliação. Há também temas emergentes, como as habilidades (*skills*) de agentes de programação e sua avaliação. O passe também expôs ruído que a triagem deve excluir: PRs automáticos de atualização de dependências, alertas de vulnerabilidade de pacotes, *changelogs* e anúncios de vagas. A curva de saturação do catálogo estabiliza em 50 códigos no lote 116 de 117. Como o catálogo é fechado, essa curva só confirma a cobertura das entradas conhecidas; a saturação que conta é a dos códigos da codificação manual.

4.5 QP4 e QP5: Prévia

Os modos de falha cobertos (QP4) e a transferência de RAG para agentes (QP5) exigem a codificação manual. Na base de detecções, 30 das 34 ferramentas vistas em unidades de agentes (88%) também aparecem em unidades de RAG (Jaccard de 0,83), e os 10 métodos são compartilhados. Apenas AgentDojo, continuous-eval, Guardrails AI e PyRIT aparecem só em agentes. Esse valor está inflado: o rótulo de sistema vem do casamento de conceitos, e muitas unidades recebem os dois rótulos. A comparação válida usará o sistema codificado.

5 DISCUSSÃO

Para praticantes. Em RAG, Ragas e DeepEval são as referências tanto no discurso quanto no uso declarado, muitas vezes juntas (*lift* de 9,6) e combinadas com conjuntos *golden* e LLMs como juízes. A pirâmide mostra, porém, que poucas unidades as mantêm rodando em CI. Já os *benchmarks* de agentes e as sondas de segurança (Guardrails AI, garak, NeMo Guardrails, PyRIT) aparecem pouco no discurso e com frequência em CI, com a ressalva de que parte dessa adoção vem dos repositórios das próprias ferramentas (Seção 6). Isso sugere que verificações de segurança e de comportamento de agentes são mais fáceis de automatizar como portões de integração do que métricas de qualidade de resposta. As ferramentas de rastreamento acompanham os *frameworks* (LangChain com LangSmith) e não aparecem como portões de teste, o que sugere uso em observabilidade e depuração, hipótese que a codificação vai verificar.

Para pesquisadores. Metade das unidades com sinal de oráculo depende de um LLM como juiz, e só 5% registram avaliação humana,

o que reforça a necessidade de juízes validados e de protocolos que meçam a concordância entre juiz e humanos. Oráculos derivados, como relações metamórficas, aparecem em 4% das unidades, apesar de o problema do oráculo ser o desafio central desses sistemas: há espaço para técnicas que reduzam a dependência de juízes e de conjuntos *golden* caros de manter. A avaliação de trajetória ainda é rara (21 unidades), embora boa parte das falhas de agentes esteja na trajetória.

Para criadores de ferramentas. A concordância negativa entre uso declarado e adoção ($\tau_b = -0{,}38$) indica que ser importado não basta para entrar no fluxo de CI. Custo de execução, não determinismo e dependência de chaves de API são barreiras plausíveis. Modos determinísticos, *cache* de respostas e orçamentos de execução podem aproximar as bibliotecas de avaliação do uso como portão de qualidade.

6 AMEAÇAS À VALIDADE

Validade de construto. "Mais usada" foi medida pela invocação em CI de repositórios públicos, o que exclui o uso privado e corporativo. Menção não é uso, e popularidade não é qualidade [4]. O critério de N3 (dois meses de commits e dois contribuidores) aproxima a adoção sustentada e pode excluir projetos legítimos com um único mantenedor. Nesta execução, o N3 não separa o repositório da própria ferramenta. A análise preliminar indica que parte das invocações em CI de *benchmarks* de agentes vem dos próprios repositórios das ferramentas e de derivados de pesquisa, o que pode inflar a adoção dessa família. A triagem manual e o script de análise, que separa esses repositórios como *self*, mitigam esse viés. O tipo de oráculo vem do método detectado, e não da codificação.

Validade interna. O catálogo orienta busca e extração e favorece ferramentas já conhecidas; o passe de descoberta mitiga esse viés. A coleta trouxe no máximo 10 itens por consulta, e a busca do GitHub ordena por relevância e trunca, o que faz do corpus uma amostra, não um censo. As contagens por ferramenta vêm do relatório da execução, que lista as ferramentas com adoção sustentada e as mais frequentes. As demais têm até 9 unidades e nenhuma adoção sustentada, e sua ausência pouco altera as correlações. As detecções ainda não foram validadas: a amostra de precisão de 200 detecções aguarda julgamento. Os autores acompanharam coletas preliminares, e por isso as proposições são exploratórias.

Validade externa. São seis fontes; Reddit, X, Discord e LinkedIn, que concentram parte da discussão de praticantes, ficaram de fora por restrições de acesso. O GitHub responde por 84% dos itens e o inglês domina. A janela vai de 01/2023 a 08/2026.

Validade de conclusão. As correlações de Kendall usam 24 ferramentas e são reportadas com seus valores-p, sem intervalos por *bootstrap*, que dependem das unidades do pacote. Pares de coocorrência com poucas unidades (como garak e *red teaming*, com 3) são indicativos.

Confiabilidade. O protocolo, o catálogo, os relatórios da execução e os scripts de análise estão versionados, e a concordância entre avaliadores será reportada por κ. As métricas do próprio minerador (revocação, precisão e estabilidade) estão no trabalho que o descreve [12].

7 TRABALHOS RELACIONADOS

Surveys recentes organizam a avaliação de LLMs [6], de sistemas RAG [35] e de agentes [34] a partir da literatura acadêmica e de *benchmarks*, e Zhang et al. [36] sistematizam o teste de sistemas de aprendizado de máquina. Este estudo é complementar: parte da prática observável e mede a força da evidência de adoção. Barnett et al. [1] derivam pontos de falha de RAG de estudos de caso, e Cemri et al. [5], uma taxonomia de falhas de sistemas multiagentes a partir de execuções; a QP4 confrontará essas taxonomias com as falhas relatadas pela comunidade. O precedente metodológico mais próximo é o de Raulamo-Jurvanen et al. [31], que usaram literatura cinza para identificar as ferramentas de automação de teste preferidas pelos praticantes. Acrescentamos a separação entre menção, uso declarado e adoção em repositórios, motivada pela distância entre popularidade e uso [4].

8 CONCLUSÃO

Este artigo investigou como a comunidade testa sistemas RAG e agentes de LLM, combinando literatura cinza e mineração de repositórios com três níveis de evidência. Discurso e prática divergem. As bibliotecas de avaliação de RAG, com Ragas e DeepEval à frente, dominam as menções e o uso declarado. A adoção sustentada em CI concentra-se em *benchmarks* de agentes e em ferramentas de segurança. As ferramentas de rastreamento são muito citadas e não têm adoção sustentada. LLM como juiz é o método dominante, e oráculos derivados e humanos são raros.

Os próximos passos são:

- completar a triagem em duas trilhas e a codificação com dupla avaliação, o que permitirá testar H2 e responder à QP4 e à QP5 com o sistema codificado;
- executar a análise de engajamento (QP3);
- repetir todas as análises na execução oficial congelada do protocolo, com o repositório da própria ferramenta separado da adoção.

DISPONIBILIDADE DE ARTEFATOS

O protocolo, o catálogo de ferramentas e métodos, o minerador e o script de análise (`analysis/artigo2.py`) estão no repositório do projeto[1], sob a licença Apache-2.0. O pacote de reprodução do corpus será depositado no Zenodo sob CC BY 4.0, com metadados, URLs e campos extraídos. O texto integral só é mantido quando a licença da fonte permite, e os autores dos itens são pseudonimizados.

AGRADECIMENTOS

A definir.

REFERÊNCIAS

[1] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. 2024. Seven Failure Points When Engineering a Retrieval Augmented Generation System. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN)*. ACM, 194–199. https://doi.org/10.1145/3644815.3644945

[2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525. https://doi.org/10.1109/TSE.2014.2372785

[3] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet Allocation. *Journal of Machine Learning Research* 3 (2003), 993–1022.

[1] https://github.com/Icar0S/Miner-Tests-Agents-Rag

[4] Hudson Borges and Marco Tulio Valente. 2018. What's in a GitHub Star? Understanding Repository Starring Practices in a Social Coding Platform. *Journal of Systems and Software* 146 (2018), 112–129. https://doi.org/10.1016/j.jss.2018.09.016

[5] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. 2025. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657.

[6] Yupeng Chang, Xu Wang, Jindong Wang, et al. 2024. A Survey on Evaluation of Large Language Models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 39:1–39:45. https://doi.org/10.1145/3641289

[7] Norman Cliff. 1993. Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. *Psychological Bulletin* 114, 3 (1993), 494–509. https://doi.org/10.1037/0033-2909.114.3.494

[8] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. https://doi.org/10.1177/001316446002000104

[9] Confident AI. 2025. DeepEval: The LLM Evaluation Framework. https://github.com/confident-ai/deepeval.

[10] Daniela S. Cruzes and Tore Dybå. 2011. Recommended Steps for Thematic Synthesis in Software Engineering. In *Proceedings of the International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 275–284. https://doi.org/10.1109/ESEM.2011.36

[11] Ícaro Santos de Oliveira, João Matheus de Lima Alves, and Ismayle de Sousa Santos. 2026. MSR-Kit: Minerador de Literatura Cinza por APIs Oficiais. https://github.com/Icar0S/Miner-Tests-Agents-Rag.

[12] Ícaro Santos de Oliveira, João Matheus de Lima Alves, and Ismayle de Sousa Santos. 2027. MSR-Kit: Mineração Reprodutível de Literatura Cinza por APIs Oficiais. (2027). Em preparação.

[13] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. Curran Associates.

[14] Leon Derczynski, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. 2024. garak: A Framework for Security Probing Large Language Models. arXiv:2406.11036.

[15] Olive Jean Dunn. 1964. Multiple Comparisons Using Rank Sums. *Technometrics* 6, 3 (1964), 241–252. https://doi.org/10.1080/00401706.1964.10490181

[16] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

[17] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*. Association for Computational Linguistics.

[18] Yunfan Gao, Yun Xiong, Xinyu Gao, et al. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

[19] Vahid Garousi, Michael Felderer, and Mika V. Mäntylä. 2019. Guidelines for Including Grey Literature and Conducting Multivocal Literature Reviews in Software Engineering. *Information and Software Technology* 106 (2019), 101–121. https://doi.org/10.1016/j.infsof.2018.09.006

[20] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[21] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.

[22] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. 2014. The Promises and Perils of Mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*. ACM, 92–101. https://doi.org/10.1145/2597073.2597074

[23] Maurice G. Kendall. 1938. A New Measure of Rank Correlation. *Biometrika* 30, 1/2 (1938), 81–93. https://doi.org/10.1093/biomet/30.1-2.81

[24] William H. Kruskal and W. Allen Wallis. 1952. Use of Ranks in One-Criterion Variance Analysis. *J. Amer. Statist. Assoc.* 47, 260 (1952), 583–621. https://doi.org/10.1080/01621459.1952.10483441

[25] J. Richard Landis and Gary G. Koch. 1977. The Measurement of Observer Agreement for Categorical Data. *Biometrics* 33, 1 (1977), 159–174. https://doi.org/10.2307/2529310

[26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. Curran Associates.

[27] Xiao Liu, Hao Yu, Hanchen Zhang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR)*.

[28] Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, et al. 2021. The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews. *BMJ* 372 (2021), n71. https://doi.org/10.1136/bmj.n71

[29] Promptfoo. 2025. promptfoo: Test Your Prompts, Agents, and RAGs. https://github.com/promptfoo/promptfoo.

[30] Paul Ralph et al. 2020. ACM SIGSOFT Empirical Standards. arXiv:2010.03525.

[31] Päivi Raulamo-Jurvanen, Mika Mäntylä, and Vahid Garousi. 2017. Choosing the Right Test Automation Tool: A Grey Literature Review of Practitioner Sources. In *Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 21–30. https://doi.org/10.1145/3084226.3084252

[32] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. https://doi.org/10.1080/01621459.1927.10502953

[33] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.

[34] Asaf Yehudai, Lilach Eden, Alan Li, et al. 2025. Survey on Evaluation of LLM-based Agents. arXiv:2503.16416.

[35] Hao Yu, Aoran Gan, Kai Zhang, et al. 2024. Evaluation of Retrieval-Augmented Generation: A Survey. arXiv:2405.07437.

[36] Jie M. Zhang, Mark Harman, Lei Ma, and Yang Liu. 2022. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Transactions on Software Engineering* 48, 1 (2022), 1–36. https://doi.org/10.1109/TSE.2019.2962027

[37] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, Vol. 36. Curran Associates.

[38] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations (ICLR)*.